RSA (Rivest-Shamir-Adleman)

Aim

To implement RSA, a public-key cipher whose security rests on the difficulty of factoring the product of two large primes. A keypair is derived from two primes `p` and `q`: the public key `(e, n)` is shared, the private key `(d, n)` is kept secret. A message is encrypted with the public key as `c = m^e mod n` and only the matching private key recovers it as `m = c^d mod n`. This demonstrator works on real integer arithmetic (BigInt) and encrypts one character code point per block.

Algorithm

Key generation is a one-time setup that produces the keypair used by encrypt and decrypt.

```
1 Function KeyGen(p, q):
2     n ← p * q
3     phi ← (p - 1) * (q - 1)                          // Euler's totient of n
4     choose e with 1 < e < phi and gcd(e, phi) = 1     // public exponent
5     d ← modular inverse of e mod phi                  // via extended Euclid
6     return public (e, n) and private (d, n)
```

```
1 Function Encrypt(plain_text, e, n):
2     cipher ← empty list
3     foreach character ch in plain_text do
4         m ← code point of ch
5         if m ≥ n then
6             error                                     // block too large; use bigger primes
7         append (m raised to e) mod n to cipher        // fast modular exponentiation
8     return cipher
```

```
1 Function Decrypt(cipher, d, n):
2     plain ← ""
3     foreach value c in cipher do
4         m ← (c raised to d) mod n
5         append character with code point m to plain
6     return plain
```

Output

Key generation — primes `p = 17`, `q = 11`

Quantity	Computation	Value
n	p * q = 17 * 11	187
phi	(p - 1)(q - 1) = 16 * 10	160
e	smallest coprime with phi (> 1)	17
d	inverse of e mod phi	113

Check: e * d mod phi = 17 * 113 mod 160 = 1, so encryption and decryption are inverses. Public key = `(17, 187)`, private key = `(113, 187)`.

Figure 1: RSA demonstrator

Encrypt — message `Hi` with public key `(e = 17, n = 187)`

Each character's code point is raised to `e` mod `n`:

Char	Code (m)	c = (m raised to e) mod n	Cipher
H	72	(72 raised to 17) mod 187	140
i	105	(105 raised to 17) mod 187	173

Result: `Hi -> [140, 173]`

Decrypt — cipher `[140, 173]` with private key `(d = 113, n = 187)`

Each cipher value is raised to `d` mod `n` to recover the code point:

Cipher (c)	m = (c raised to d) mod n	Code	Char
140	(140 raised to 113) mod 187	72	H
173	(173 raised to 113) mod 187	105	i

Result: `[140, 173] -> Hi`

Every character code must be smaller than `n`; the primes here (n = 187) only fit code points below 187, so real use needs far larger primes. An eavesdropper who sees `(e, n)` and the cipher would have to factor `n` back into `p` and `q` to find `d`, which is infeasible for large primes.